

MT 2 Review

Contents

Study Guide: Chapter 3 – Transport Layer	5
Summary of Pages 1–10	5
Overview	5
Roadmap of Chapter 3	5
Transport Services and Protocols	5
Analogy: Transport vs. Network Layer	5
Sender and Receiver Actions	5
TCP vs. UDP.....	5
Important Note.....	6
Comprehension Q&A (Pages 1–10)	6
1. What is the main function of the transport layer? (Slide 3-2)	6
2. Name the key principles behind transport layer services. (Slide 3-2)	6
3. What are the two main Internet transport protocols, and how do they differ? (Slides 3-4, 3-9).....	6
4. Explain the household analogy for transport and network layers. (Slides 3-5, 3-6) ...	6
5. What is the difference between the network layer and the transport layer? (Slide 3-6)	6
6. What are the sender and receiver actions at the transport layer? (Slides 3-7, 3-8) ...	7
7. What features does TCP provide that UDP does not? (Slide 3-9).....	7
8. Do TCP or UDP provide delay or bandwidth guarantees? (Slide 3-9)	7
Summary of Pages 11–20	7
Multiplexing and Demultiplexing (Slides 3-11 to 3-16)	7
Demultiplexing in UDP (Slides 3-17 to 3-19)	7
Demultiplexing in TCP (Slide 3-20)	7
Comprehension Q&A (Pages 11–20)	8
1. (Slide 3-16) What is multiplexing at the sender, and what is demultiplexing at the receiver?	8
2. (Slide 3-17) What information does a transport-layer segment use to deliver data to the correct socket?	8

3. (Slide 3-18) How does UDP demultiplexing work when multiple datagrams arrive with the same destination port but different source addresses?	8
4. (Slide 3-19) In the UDP example, how are datagrams from different clients delivered to the server?	8
5. (Slide 3-20) What is the 4-tuple used to identify a TCP socket, and why is it important?	8
6. (Slide 3-20) How can a server support multiple simultaneous TCP connections from different clients?	8
Summary of Pages 21–30	9
Connection-Oriented Demultiplexing	9
Multiplexing/Demultiplexing Recap	9
UDP: User Datagram Protocol	9
UDP Actions	9
UDP Segment Header	9
Comprehension Q&A (Pages 21–30)	9
Summary of Pages 31–40	10
UDP Checksum and Internet Checksum	10
UDP Recap	11
Reliable Data Transfer: Stop-and-Wait	11
Pipelined Protocols	11
Go-Back-N Protocol	11
Go-Back-N Receiver	11
Comprehension Q&A (Pages 31–40) – Answers & Corrections	11
Summary of Pages 41–50	13
Go-Back-N in Action	13
Selective Repeat Protocol	13
Selective Repeat Dilemma	13
TCP Overview	13
TCP Segment Structure	13
Comprehension Q&A (Pages 41–50) – Answers & Corrections	14
Important Notes (Pages 41–50)	14
Summary of Pages 51–60	15
TCP Sequence Numbers and ACKs	15
TCP Round Trip Time (RTT) and Timeout	15

TCP Sender (Simplified)	15
TCP Receiver: ACK Generation	15
TCP Retransmission Scenarios	15
TCP Fast Retransmit.....	15
Storage Note.....	16
Comprehension Q&A (Pages 51–60) – Answers & Corrections	16
Summary of Pages 61–70	16
TCP Flow Control	16
TCP Connection Management	17
TCP 2-Way Handshake	17
Comprehension Q&A (Pages 61–70) – Answers & Corrections	17
Summary of Pages 71–80	18
TCP 2-Way Handshake Problems	18
TCP 3-Way Handshake	18
Closing a TCP Connection	18
Principles of Congestion Control.....	18
Causes/Costs of Congestion (Scenarios).....	19
Comprehension Q&A (Pages 71–80) – Answers & Corrections	19
Important Note (Pages 71–80).....	20
Summary of Pages 81–90	20
Causes and Costs of Congestion (Scenarios 2 & 3).....	20
Approaches to Congestion Control	20
Comprehension Q&A (Pages 81–90) – Answers & Corrections	20
Summary of Pages 91–100	21
TCP Congestion Control: AIMD and Beyond	21
Comprehension Q&A (Pages 91–100) – Answers & Corrections.....	22
Important Notes (Pages 91–100)	23
Summary of Pages 101–110	23
Delay-Based and Explicit Congestion Control; Fairness; QUIC	23
Comprehension Q&A (Pages 101–110) – Answers & Corrections	23
Summary of Pages 111–121	24
QUIC, Go-Back-N, TCP FSMs, and Throughput	24
Comprehension Q&A (Pages 111–121) – Answers & Corrections	25

Important Note (Pages 111–121)	25
4	25
Summary of Chapter 4, Pages 1–29.....	25
Network Layer: Data Plane Overview	25
Comprehension Q&A (Chapter 4, Pages 1–29) – Answers & Corrections	26
Important Notes (Chapter 4, Pages 1–29).....	27

Study Guide: Chapter 3 – Transport Layer

Summary of Pages 1–10

Overview

- The transport layer provides logical communication between application processes running on different hosts.
- Key principles: multiplexing/demultiplexing, reliable data transfer, flow control, congestion control.
- Main Internet transport protocols: UDP (connectionless), TCP (connection-oriented, reliable).

Roadmap of Chapter 3

- Transport-layer services
- Multiplexing and demultiplexing
- UDP: connectionless transport
- Principles of reliable data transfer
- TCP: connection-oriented transport
- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality

Transport Services and Protocols

- Transport protocols operate in end systems (hosts), breaking application messages into segments (sender) and reassembling them (receiver).
- Two main protocols: TCP and UDP.

Analogy: Transport vs. Network Layer

- Hosts = houses, processes = kids, messages = letters, transport protocol = parents demultiplexing, network protocol = postal service.
- Network layer: communication between hosts; transport layer: communication between processes.

Sender and Receiver Actions

- Sender: receives application message, creates segment, passes to IP.
- Receiver: receives segment from IP, checks header, extracts message, delivers to application.

TCP vs. UDP

- TCP: reliable, in-order delivery, congestion and flow control, connection setup.
- UDP: unreliable, unordered delivery, minimal overhead.

- Neither provides delay or bandwidth guarantees.
-

Important Note

Receiver: IP address, MAC, port-# → MUX, Net.Addr.Transl. (saves your router address), Load balancing/sharing... new system that makes sure 1 line does not carry all the burden → data ≥ sum of all inputs (with a safety factor of 4 or 10) → DeMUX

- $R_{out} \geq n \sum_{i=1}^n \{ R_i \}$

Comprehension Q&A (Pages 1–10)

1. What is the main function of the transport layer? (Slide 3-2)

Your answer: transport layer sends logical communication to the other users. **Correction:** The transport layer provides logical communication between application processes running on different hosts.

2. Name the key principles behind transport layer services. (Slide 3-2)

Your answer: Multiplexing, demuxing, UDP, TCP, logical communication, segmentation
Correction: Multiplexing/demultiplexing, reliable data transfer, flow control, congestion control.

3. What are the two main Internet transport protocols, and how do they differ? (Slides 3-4, 3-9)

Your answer: TCP and UDP. TCP is with connection, UDP is connectionless. **Correction:** TCP (connection-oriented, reliable, provides flow and congestion control) and UDP (connectionless, unreliable, minimal overhead).

4. Explain the household analogy for transport and network layers. (Slides 3-5, 3-6)

Your answer: Network sends packets to the entire host (i.e. letters to the house), whereas Transport layer sends packets to individual processes (i.e. letters to inhabitants)
Correction: Correct.

5. What is the difference between the network layer and the transport layer? (Slide 3-6)

Your answer: see answer for question 4. **Correction:** The network layer provides logical communication between hosts; the transport layer provides logical communication between processes on those hosts.

6. What are the sender and receiver actions at the transport layer? (Slides 3-7, 3-8)

Your answer: Sender is passed application message & creates segment & header values to be sent to IP; Receiver checks header, extracts application layer message, and demuxes the message to application via socket. **Correction:** Correct.

7. What features does TCP provide that UDP does not? (Slide 3-9)

Your answer: handshaking **Correction:** TCP provides reliable, in-order delivery, congestion control, flow control, and connection setup (handshaking is part of connection setup).

8. Do TCP or UDP provide delay or bandwidth guarantees? (Slide 3-9)

Your answer: they provide neither **Correction:** Correct.

Summary of Pages 11–20

Multiplexing and Demultiplexing (Slides 3-11 to 3-16)

- Multiplexing: Sender handles data from multiple sockets, adds transport headers for later demultiplexing.
- Demultiplexing: Receiver uses header info (IP addresses and port numbers) to deliver received segments to the correct socket/process.
- Each transport-layer segment contains source/destination IP addresses and port numbers.

Demultiplexing in UDP (Slides 3-17 to 3-19)

- UDP sockets are identified by destination port number only.
- Multiple datagrams with the same destination port but different source IP/port can be delivered to the same socket.
- Example: Several clients can send to the same UDP server port, and all are delivered to the same server socket.

Demultiplexing in TCP (Slide 3-20)

- TCP sockets are identified by a 4-tuple: source IP, source port, destination IP, destination port.
 - Server can support many simultaneous TCP sockets, each associated with a different client (unique 4-tuple).
 - Receiver uses all four values to direct the segment to the correct socket.
-

Comprehension Q&A (Pages 11–20)

1. (Slide 3-16) What is multiplexing at the sender, and what is demultiplexing at the receiver?

Your answer: Muxing at sender adds header that is used by the receiver when returning message **Correction:** Multiplexing at the sender handles data from multiple sockets and adds a transport header (not just for returning messages, but for all outgoing segments). Demultiplexing at the receiver uses the header info to deliver received segments to the correct socket/process.

2. (Slide 3-17) What information does a transport-layer segment use to deliver data to the correct socket?

Your answer: IP address, src-port, dst-port **Correction:** Correct! (IP addresses and port numbers are used.)

3. (Slide 3-18) How does UDP demultiplexing work when multiple datagrams arrive with the same destination port but different source addresses?

Your answer: I don't get this one... **Correction:** All datagrams with the same destination port are delivered to the same socket, regardless of their source IP or source port.

4. (Slide 3-19) In the UDP example, how are datagrams from different clients delivered to the server?

Your answer: the packet always includes the source and destination port #s **Correction:** Correct! The server receives all datagrams sent to its port, regardless of the client's source port or address.

5. (Slide 3-20) What is the 4-tuple used to identify a TCP socket, and why is it important?

Your answer: 4-tuple includes scr-IP add, scr-port-#, dst-IP addr, dst-port-#. All 4 are important and used by the deMUXer to direct segment to appropriate socket **Correction:** Correct!

6. (Slide 3-20) How can a server support multiple simultaneous TCP connections from different clients?

Your answer: because each socket AND each client now have their own address in a unique 4-tuple. making each sender/receiver unique. **Correction:** Correct!

Summary of Pages 21–30

Connection-Oriented Demultiplexing

- TCP uses a 4-tuple (source IP, source port, destination IP, destination port) to demultiplex incoming segments to the correct socket.
- Multiple segments destined for the same server port (e.g., port 80) but from different clients are directed to different sockets.

Multiplexing/Demultiplexing Recap

- Multiplexing and demultiplexing occur at all layers, but at the transport layer:
 - UDP uses only the destination port number for demultiplexing.
 - TCP uses the full 4-tuple.

UDP: User Datagram Protocol

- UDP is a simple, connectionless, “best effort” protocol.
- Segments may be lost or delivered out of order.
- No handshaking or connection state.
- Small header size, no congestion control.
- Used for applications like streaming media, DNS, SNMP, and HTTP/3.
- If reliability is needed, it must be implemented at the application layer.

UDP Actions

- **Sender:** Receives application message, determines header values, creates segment, passes to IP.
- **Receiver:** Receives segment from IP, checks checksum, extracts message, delivers to application via socket.

UDP Segment Header

- Contains source port, destination port, length, and checksum.
- Application data (payload) follows the header.

Slide Images (Pages 21–30)

Slides 21–30 are available as PNGs in the pdf_outputs folder: slide-21.png, slide-22.png, ..., slide-30.png

Comprehension Q&A (Pages 21–30)

1. What 4 values does TCP use to demultiplex incoming segments to the correct socket? (Slide 3-21) **Your answer:** scr-IP add, scr-port-#, dst-IP addr, dst-port-#, with destination port being 80. **Correction:** Correct! TCP uses the source IP

address, source port number, destination IP address, and destination port number (the 4-tuple).

2. Why is UDP considered a “best effort” protocol? (Slide 3-24) **Your answer:** If it sends and loses part or all of the packet, there is no way to confirm that it has been received. **Correction:** Correct! UDP does not guarantee delivery, order, or error recovery—segments may be lost or delivered out of order, and there is no acknowledgment.
3. Name two reasons why an application might choose UDP over TCP. (Slide 3-24) **Your answer:** if it uses simplex or half-duplex communication. Or if that app needs very fast communication, and can proceed with some data loss (ie, streaming). **Correction:** Partially correct. The main reasons are: (1) No connection establishment, so it’s faster and has lower latency; (2) Small header size and no congestion control, allowing applications to send data as fast as needed (useful for streaming, DNS, etc.); (3) Tolerant of some data loss.
4. What must an application do if it needs reliable transfer over UDP? (Slide 3-25) **Your answer:** add needed reliability at application layer (e.g. HTTP/3), and add congestion control at application layer **Correction:** Correct! Reliability and congestion control must be implemented at the application layer if needed.
5. What are the main fields in a UDP segment header? (Slide 3-30) **Your answer:** src-port#, dst-port#, length, checksum, and message/app-data **Correction:** Correct! The UDP header includes source port number, destination port number, length, and checksum. The message/app-data is the payload, not part of the header.
6. Describe the sender and receiver actions for UDP at the transport layer. (Slides 3-27 to 3-29) **Your answer:** (Not answered) **Correction:**
 - **Sender:** Receives application message, determines UDP header values, creates UDP segment, passes segment to IP.
 - **Receiver:** Receives segment from IP, checks UDP checksum, extracts application-layer message, delivers to application via socket.

Summary of Pages 31–40

UDP Checksum and Internet Checksum

- The UDP checksum detects errors (bit flips) in transmitted segments.
- Sender computes a checksum over the segment (header + data), receiver recomputes and compares.

- Internet checksum uses one's complement sum of 16-bit words; a carry-out from the most significant bit is wrapped around.
- The checksum is not foolproof—some errors may go undetected.

UDP Recap

- UDP is a “no frills” protocol: segments may be lost or delivered out of order.
- No setup/handshaking, can function even if network service is compromised.
- Reliability can be improved at the application layer (e.g., HTTP/3).

Reliable Data Transfer: Stop-and-Wait

- Sender transmits one packet, waits for ACK before sending the next.
- Performance is limited by round-trip time (RTT) and bandwidth.

Pipelined Protocols

- Sender can transmit multiple packets before needing ACKs (increases utilization).
- Requires larger sequence number space and buffering.

Go-Back-N Protocol

- Sender maintains a window of up to N unacknowledged packets.
- Uses cumulative ACKs: ACK(n) acknowledges all packets up to n.
- On timeout, retransmits packet n and all higher sequence numbers in the window.

Go-Back-N Receiver

- Always sends ACK for the highest in-order packet received.
- May discard or buffer out-of-order packets.
- Duplicate ACKs may be generated.

Slide Images (Pages 31–40)

Slides 31–40 are available as PNGs in the pdf_outputs folder:
slide-31.png, slide-32.png, ..., slide-40.png

Comprehension Q&A (Pages 31–40) – Answers & Corrections

1. What is the purpose of the UDP checksum, and how does it work? (Slides 3-31 to 3-33) **Your answer:** checksum works by adding a 1's complement (11010101....) to the segment, and iterating a wrap-around of the MSB's carry-over to the back, until you arrive at the sum. the “sum” should be the XOR of the “checksum” **Correction:** The UDP checksum is calculated by taking the one's complement sum of all 16-bit words in the segment (header and data), with any carry-out from the most significant bit wrapped around. The receiver recomputes the checksum and compares it to the value in the segment. If they match, the segment is assumed correct; otherwise, an error is detected. (Note: The sum is not an XOR, but a one's complement addition.)

2. Why is the Internet checksum considered weak protection? (Slide 3-34) **Your answer:** because, even if 2 digits have been swapped via bit error, as long as both the segment and checksum experienced the same bits/digits swapped, the checksum will not notice the difference. **Correction:** Correct in spirit. The Internet checksum can fail to detect some errors, such as certain bit swaps or multiple bit errors that cancel each other out, so it is not foolproof.
3. List two advantages and one disadvantage of using UDP. (Slide 3-35) **Your answer:** ADV: no handshake and so no RTT, and can be reliable with checksum, but even more reliable with application layer checks (HTTP/3). DIS: segments may be lost or delivered out of order. **Correction:** Correct. Advantages: (1) No connection setup/handshaking (no RTT delay), (2) Simple protocol with low overhead. Disadvantage: Segments may be lost, duplicated, or delivered out of order (unreliable).
4. Describe the stop-and-wait protocol and its main performance limitation. (Slide 3-36) **Your answer:** send only one packet at a time, and don't send the next until ACK is given back. This has very slow performance. **Correction:** Correct. The sender transmits one packet and waits for an acknowledgment before sending the next. The main limitation is low utilization of the link, especially when the round-trip time (RTT) is large compared to the transmission time.
5. What is pipelining in reliable data transfer protocols, and why is it useful? (Slide 3-37) **Your answer:** Send a bunch of packets at once, and Rx sends an ACK, with a bunch of "yet to be acknowledged" packets in the same return pipeline. **Correction:** Correct. Pipelining allows the sender to transmit multiple packets before needing acknowledgments, increasing link utilization and throughput.
6. How does the Go-Back-N protocol improve efficiency over stop-and-wait? (Slides 3-39 to 3-40) **Your answer:** It creates a window of receiving N packets, and proceeds to the next window only when all of those packets have been ACK'd **Correction:** Correct. Go-Back-N allows the sender to have up to N unacknowledged packets in the pipeline, improving efficiency by keeping the link busy. On a timeout, the sender retransmits the unacknowledged packets in the window.
7. What does the Go-Back-N receiver do when it receives an out-of-order packet? (Slide 3-40) **Your answer:** it can discard, buffer, or re-ACK pkt with the highest in-order sequence-#N **Correction:** The receiver typically discards out-of-order packets (though buffering is possible in some implementations) and re-ACKs the highest in-order sequence number received so far.
 1. S + w
 2. Go back N
 3. Select repeat

Summary of Pages 41–50

Go-Back-N in Action

- Sender maintains a window of N unacknowledged packets.
- On timeout, retransmits the timed-out packet and all subsequent packets in the window.
- Receiver only accepts in-order packets and sends cumulative ACKs for the last in-order packet received.

Selective Repeat Protocol

- Receiver individually acknowledges all correctly received packets and buffers out-of-order packets for in-order delivery.
- Sender maintains a timer for each unacknowledged packet and retransmits only those that time out.
- Both sender and receiver maintain windows of size N.
- More efficient than Go-Back-N, but requires more complex logic and buffering.

Selective Repeat Dilemma

- If the sequence number space is too small relative to the window size, ambiguity can occur (e.g., receiver may accept a retransmitted packet as new data).
- To avoid this, the sequence number space must be at least twice the window size.

TCP Overview

- TCP is point-to-point (one sender, one receiver), reliable, and provides in-order byte stream delivery.
- Full duplex: data can flow in both directions simultaneously.
- Uses cumulative ACKs and pipelining (window-based).
- Connection-oriented: uses handshaking to establish connection state.
- Flow controlled: sender will not overwhelm receiver.

TCP Segment Structure

- TCP segment includes source/destination ports, sequence number, acknowledgment number, header length, flags (SYN, ACK, FIN, etc.), receive window, checksum, urgent pointer, options, and application data.
- Sequence numbers count bytes, not segments.

Slide Images (Pages 41–50)

Slides 41–50 are available as PNGs in the pdf_outputs folder:
slide-41.png, slide-42.png, ..., slide-50.png

Comprehension Q&A (Pages 41–50) – Answers & Corrections

1. In Go-Back-N, what happens when a packet times out? (Slide 3-41) **Your answer:** once the packet N times-out, it resends every packet starting from N, and continues like this, always looking for the ACKs **Correction:** Correct. When a packet times out, the sender retransmits the timed-out packet and all subsequent packets in the window (i.e., all unacknowledged packets starting from N).
2. How does Selective Repeat differ from Go-Back-N in handling out-of-order packets? (Slides 3-42 to 3-45) **Your answer:** It has 4 different types of ACK statuses (already acked; sent, not yet acked; usable, not yet sent; not usable). this allows the Tx to only resend the unACK'd packets, instead of resending packets that were already ACK'd, like in Go-Back-N. **Correction:** Correct in spirit. Selective Repeat allows the receiver to individually acknowledge and buffer out-of-order packets, and the sender only retransmits those specific packets that time out, rather than all unacknowledged packets.
3. What is the key requirement for the relationship between sequence number space and window size in Selective Repeat? (Slide 3-47) **Your answer:** the relationship needed b/w seq#-size and window size is that they cannot be the same. **Correction:** The sequence number space must be at least twice the window size to avoid ambiguity and ensure correct operation.
4. List three key features of TCP as described in the overview. (Slide 3-49) **Your answer:** Point-to-point, full-duplex data, cumulative ACKs **Correction:** Correct. Other features include reliable, in-order byte stream, connection-oriented, flow controlled, and pipelined.
5. What are the main fields in a TCP segment header? (Slide 3-50) **Your answer:** src-port#, dst-port#, sequence number, ACK number, Length, (C,E) congestion notification, RST/SYN/FIN, receive window, checksum, TCP options, message/data **Correction:** Correct. The main fields are source port, destination port, sequence number, acknowledgment number, header length, flags (C, E, RST, SYN, FIN, etc.), receive window, checksum, urgent pointer, options, and application data.

Important Notes (Pages 41–50)

- Selective Repeat: // receiver decides speed, because receiver is the bottle-neck/limiting factor
- S.R. dilemma:
 - For exam: where are the errors? What did come thru? What didn't?
 - answer: they all made it thru here, cuz the speed isn't that fast

Summary of Pages 51–60

TCP Sequence Numbers and ACKs

- Sequence numbers identify the byte stream number of the first byte in a segment's data.
- ACKs indicate the next expected byte (cumulative acknowledgment).
- TCP does not specify how to handle out-of-order segments; it's up to the implementor.

TCP Round Trip Time (RTT) and Timeout

- Timeout should be set longer than RTT, but RTT varies.
- Too short: unnecessary retransmissions; too long: slow loss recovery.
- EstimatedRTT is calculated using an exponential weighted moving average (EWMA) of recent SampleRTT values.
- $\text{TimeoutInterval} = \text{EstimatedRTT} + 4 * \text{DevRTT}$, where DevRTT is the EWMA of the deviation of SampleRTT from EstimatedRTT.
- Uses some moving average method to detect traffic flow-rate.
- A is a fudge factor made over 20+ years of gathering data (feedback loop equation).
- All parts of the formula will be provided; your job is to learn, not memorize.

TCP Sender (Simplified)

- On data from application: create segment, assign sequence number, start timer if not running.
- On timeout: retransmit the segment that caused the timeout, restart timer.
- On ACK: update ACKed segments, restart timer if unACKed segments remain.

TCP Receiver: ACK Generation

- In-order segment: may delay ACK up to 500ms for possible next segment; otherwise, send cumulative ACK.
- Out-of-order segment: send duplicate ACK indicating the next expected byte.
- Filling a gap: send ACK immediately if the segment fills the lower end of the gap.
- Just understand the purpose of a cumulative ACK; don't worry about the speed.

TCP Retransmission Scenarios

- Lost ACKs and premature timeouts can cause retransmissions.
- Cumulative ACKs can cover for lost ACKs.

TCP Fast Retransmit

- If sender receives three duplicate ACKs for the same data, it retransmits the unACKed segment with the smallest sequence number (triple duplicate ACKs).
- Fast retransmit avoids waiting for a timeout when a segment is likely lost.

Storage Note

- HDDs can get corrupted; SSDs don't corrupt in the same way, but after enough read/writes, SSDs just die.
- SSD: The MTBF (mean time between failures) is defined; all ASICs have semiconductors with limited life; much faster data access.

Slide Images (Pages 51–60)

Slides 51–60 are available as PNGs in the pdf_outputs folder:
slide-51.png, slide-52.png, ..., slide-60.png

Comprehension Q&A (Pages 51–60) – Answers & Corrections

1. What does a TCP sequence number represent? (Slide 3-51) **Your answer:** Byte stream's number of the first byte in segment data **Correction:** Correct! The sequence number is the byte-stream number of the first byte in the segment's data.
2. How does TCP estimate round-trip time (RTT) and set its timeout interval? (Slides 3-53 to 3-55) **Your answer:** SampleRTT: measured time from segment transmission until ACK receipt **Correction:** Correct in part. TCP measures SampleRTT (the time from segment transmission to ACK receipt), then uses an exponential weighted moving average (EWMA) to compute EstimatedRTT. TimeoutInterval is set as EstimatedRTT plus 4 times the deviation (DevRTT) to provide a safety margin.
3. What happens at the TCP sender when a timeout occurs? (Slide 3-56) **Your answer:** retransmit segment that caused timeout, then restart timer **Correction:** Correct! The sender retransmits the segment that caused the timeout and restarts the timer.
4. How does the TCP receiver generate ACKs for in-order and out-of-order segments? (Slide 3-57) **Your answer:** in-order: single cumulative ACK, ACKing both in-order segments. out-of-order: immediately send duplicate ACK, indicating seq. # of next expected byte **Correction:** Correct! For in-order segments, the receiver may delay or send a cumulative ACK. For out-of-order segments, the receiver immediately sends a duplicate ACK indicating the next expected byte.
5. What is TCP fast retransmit, and when is it triggered? (Slide 3-60) **Your answer:** only resend what wasn't ACK'd **Correction:** Not quite. TCP fast retransmit is triggered when the sender receives three duplicate ACKs for the same data, indicating a segment was likely lost. The sender immediately retransmits the missing segment without waiting for a timeout.

Summary of Pages 61–70

TCP Flow Control

- Flow control prevents the sender from overwhelming the receiver's buffer.

- The receiver “advertises” available buffer space in the rwnd (receive window) field of the TCP header.
- The sender limits the amount of unacknowledged (“in-flight”) data to the value of rwnd.
- This guarantees the receive buffer will not overflow, regardless of how fast the network delivers data.
- The receive buffer size can be set via socket options (default is often 4096 bytes), and many OSes auto-adjust it.
- **Note (Slide 65):** The NAT/router takes the local IP addresses and outputs its own stamp in the output to the network.

TCP Connection Management

- Before exchanging data, TCP uses a handshake to establish a connection.
- Both sides agree to establish the connection and on parameters (e.g., starting sequence numbers, buffer sizes).
- The connection state is set to ESTAB (established) after the handshake.
- Example code: `Socket clientSocket = newSocket("hostname", "port number");`
and `Socket connectionSocket = welcomeSocket.accept();`

TCP 2-Way Handshake

- A simple 2-way handshake involves a connection request and acceptance.
- Issues: variable delays, retransmitted or reordered messages, and the inability to “see” the other side can cause problems.
- The 2-way handshake is not always reliable in real networks.

Slide Images (Pages 61–70)

Slides 61–70 are available as PNGs in the pdf_outputs folder:
slide-61.png, slide-62.png, ..., slide-70.png

Comprehension Q&A (Pages 61–70) – Answers & Corrections

1. What is the purpose of TCP flow control, and how is it implemented? (Slides 3-62 to 3-66) **Your answer:** to make sure the receivers’ buffers are not fed with too much information at one time. **Correction:** Correct! TCP flow control ensures the sender does not overwhelm the receiver’s buffer by limiting the amount of unacknowledged data in flight to the receiver’s advertised window size (rwnd).
2. What does the receiver advertise to the sender in the TCP header, and why? (Slide 3-66) **Your answer:** ‘free’ buffer space, guaranteeing that Rx buffer will not overflow **Correction:** Correct! The receiver advertises the amount of free buffer space (the receive window, rwnd) so the sender knows how much data it can safely send without overflowing the receiver’s buffer.

3. What is the main goal of the TCP handshake process? (Slides 3-68 to 3-70) **Your answer:** to ensure that every packet is accounted for **Correction:** Partially correct. The main goal is to establish a reliable connection, ensuring both sides are ready and agree on parameters (e.g., sequence numbers, buffer sizes) before data transfer begins.
4. Why is a simple 2-way handshake not always reliable in real networks? (Slide 3-69) **Your answer:** variable delays, retransmitted messages, message re-ordering, can't "see" other side **Correction:** Correct! Variable delays, retransmissions, message reordering, and the inability to confirm the other side's state can cause problems with a simple 2-way handshake.
5. (Slide 65) What does a NAT/router do with local IP addresses when sending packets to the network? **Your answer:** i don't know **Correction:** The NAT/router replaces local (private) IP addresses with its own public IP address ("stamp") in outgoing packets, allowing multiple devices on a local network to share a single public IP address

Summary of Pages 71–80

TCP 2-Way Handshake Problems

- 2-way handshake can result in "half-open" connections (server thinks connection is open, client is gone) or duplicate data being accepted due to retransmissions and message reordering.

TCP 3-Way Handshake

- The 3-way handshake (SYN, SYN-ACK, ACK) ensures both client and server are live and agree on initial sequence numbers before data transfer.
- Example: client sends SYN, server replies with SYN-ACK, client responds with ACK (may include data).

Closing a TCP Connection

- Both client and server close their side by sending a segment with FIN=1.
- The other side responds with ACK (can be combined with its own FIN).
- Simultaneous FIN exchanges are handled.

Principles of Congestion Control

- Congestion: too many sources sending too much data too fast for the network to handle.
- Manifestations: long delays (queueing in router buffers), packet loss (buffer overflow).
- Congestion control is different from flow control (which is sender/receiver specific).
- Congestion is a top-10 problem in networking.

Causes/Costs of Congestion (Scenarios)

- Scenario 1: One router, infinite buffers, two flows, no retransmissions. As arrival rate approaches half the link capacity ($R/2$), delays increase.
- Scenario 2: One router, finite buffers, sender retransmits lost packets. Retransmissions increase the load on the network, even if application-layer input equals output.

Slide Images (Pages 71–80)

Slides 71–80 are available as PNGs in the pdf_outputs folder:
slide-71.png, slide-72.png, ..., slide-80.png

Comprehension Q&A (Pages 71–80) – Answers & Corrections

1. What is a “half-open” connection in the context of a TCP 2-way handshake? (Slide 3-71) **Your answer:** no client, only server **Correction:** Correct! A half-open connection occurs when the server believes the connection is open, but the client has already terminated or disappeared.
2. What are the steps of the TCP 3-way handshake, and what does each step accomplish? (Slide 3-73) **Your answer:** I) choose initial sequence#, send TCP-SYN message, II) same but server sends SYNACK message, III) Server indicates server is live **Correction:** Partially correct. The steps are: (1) Client sends SYN with initial sequence number, (2) Server replies with SYN-ACK (acknowledges client’s SYN and sends its own SYN with initial sequence number), (3) Client sends ACK to acknowledge server’s SYN. This ensures both sides are live and agree on initial sequence numbers.
3. How is a TCP connection closed? (Slide 3-75) **Your answer:** Tx sends TCP segment with FIN bit = 1 **Correction:** Correct! Each side closes its half of the connection by sending a segment with FIN=1. The other side responds with an ACK (which can be combined with its own FIN).
4. What is the difference between congestion control and flow control? (Slide 3-77) **Your answer:** flow control manages the flow between one sender and receiver, whereas congestion control manages the overall traffic in the network as a whole **Correction:** Correct! Flow control prevents a fast sender from overwhelming a slow receiver; congestion control prevents too much data from being injected into the network, avoiding overload.
5. In scenario 1, what happens as the arrival rate approaches half the link capacity ($R/2$)? (Slide 3-78) **Your answer:** the arrival rate out (λ_{out}) ceilings, and cannot go higher **Correction:** Correct! As the arrival rate approaches $R/2$, the output rate cannot increase further, and delays increase significantly.
6. In scenario 2, how do retransmissions affect congestion? (Slides 3-79 to 3-80) **Your answer:** retransmissions do affect congestion. so senders should have knowledge

to send only when router has buffers available **Correction:** Correct! Retransmissions increase the load on the network, potentially worsening congestion. Ideally, senders should only retransmit when there is buffer space available in the network.

Important Note (Pages 71–80)

- Optical packets cannot be stored. So a “hot potato protocol” is used to make sure fibre signals are handled straight away.

Summary of Pages 81–90

Causes and Costs of Congestion (Scenarios 2 & 3)

- In realistic networks, packets can be lost due to full router buffers, requiring retransmissions.
- Unnecessary retransmissions occur when senders time out prematurely, sending duplicate packets that may both be delivered, wasting network capacity.
- As the input rate approaches half the link capacity ($R/2$), delays increase and throughput cannot exceed the link’s capacity.
- In multi-hop, multi-sender scenarios, some flows may be completely starved (throughput approaches zero) due to congestion and buffer overflows.
- Costs of congestion include wasted transmission capacity, wasted energy, and reduced effective throughput due to retransmissions and duplicate packets.

Approaches to Congestion Control

- End-to-end congestion control: No explicit feedback from the network; endpoints infer congestion from observed loss or delay (e.g., TCP).
- Network-assisted congestion control: Routers provide direct feedback to hosts, indicating congestion or setting sending rates (e.g., TCP ECN, ATM, DECbit).

Slide Images (Pages 81–90)

Slides 81–90 are available as PNGs in the pdf_outputs folder:
slide-81.png, slide-82.png, ..., slide-90.png

Comprehension Q&A (Pages 81–90) – Answers & Corrections

1. Why do unnecessary retransmissions occur in real networks, and what is their effect on throughput? (Slides 3-82 to 3-84) **Your answer:** because of buffer delays causing the transmitter to timeout **Correction:** Correct! Unnecessary retransmissions occur when senders time out and resend packets that are actually just delayed, not lost. This wastes network capacity and reduces effective throughput.
2. What happens to some flows in a multi-hop, multi-sender congestion scenario? (Slide 3-85) **Your answer:** there is just more buffer delay, an possibility of retransmit

Correction: Partially correct. In severe congestion, some flows may be completely starved, with their throughput approaching zero due to persistent buffer overflows and dropped packets.

3. What are the main costs of congestion in a network? (Slides 3-86 to 3-87) **Your answer:** wasted packet, wasted energy. retransmission & duplicate losses.
Correction: Correct! Costs include wasted transmission capacity, wasted energy, and reduced throughput due to retransmissions and duplicate packets. Upstream resources are wasted for packets lost downstream.
4. What is the difference between end-to-end and network-assisted congestion control? (Slides 3-88 to 3-89) **Your answer:** in end-to-end CC, the terminals infer congestion from the delay/loss from data & ACK. but in network controlled CC, routers provide direct feedback of congestion. **Correction:** Correct! End-to-end congestion control infers congestion from observed loss or delay, while network-assisted control uses explicit feedback from routers to indicate congestion or set sending rates.

Summary of Pages 91–100

TCP Congestion Control: AIMD and Beyond

- TCP uses Additive Increase Multiplicative Decrease (AIMD) to probe for available bandwidth: it increases the sending rate by 1 MSS per RTT until loss is detected, then cuts the rate in half on loss.
- AIMD is a distributed, asynchronous algorithm that optimizes flow rates network-wide and provides stability.
- TCP sender uses a congestion window (cwnd) to limit the amount of unacknowledged data in the network. cwnd is dynamically adjusted based on network feedback.
- TCP starts with “slow start”: cwnd begins at 1 MSS and doubles every RTT until a loss event, then switches to linear increase (congestion avoidance).
- The switch from exponential to linear increase happens when cwnd reaches half its value before the last loss (ssthresh).
- TCP Reno and TCP Tahoe differ in how they respond to loss: Reno halves cwnd on triple duplicate ACKs, Tahoe resets cwnd to 1 MSS on timeout.
- TCP CUBIC (default in Linux) improves on AIMD by ramping up quickly to the previous maximum window, then more cautiously as it approaches that value, achieving higher throughput in many scenarios.
- The bottleneck link in the network determines the maximum achievable throughput; increasing the sending rate beyond this point only increases delay, not throughput.

Slide Images (Pages 91–100)

Slides 91–100 are available as PNGs in the pdf_outputs folder:
slide-91.png, slide-92.png, ..., slide-100.png

Comprehension Q&A (Pages 91–100) – Answers & Corrections

1. What is the AIMD approach in TCP congestion control, and why is it used? (Slides 3-91 to 3-92) **Your answer:** additive increase, multiplicative decrease. So that the network is never over taxed, and can lower in throughput very quickly **Correction:** Correct! AIMD (Additive Increase, Multiplicative Decrease) allows TCP to probe for available bandwidth by increasing the sending rate gradually and decreasing it rapidly when congestion is detected, preventing network overload and ensuring stability.
2. How does TCP's "slow start" mechanism work, and when does it switch to congestion avoidance? (Slides 3-94 to 3-95) **Your answer:** after every ACK, Tx send double the messages/packets. it switches to linear increase (no longer 2^x) when cwnd gets to half its value before timeout. **Correction:** Correct! During slow start, the congestion window (cwnd) doubles every RTT (exponential growth) as each ACK is received. It switches to linear increase (congestion avoidance) when cwnd reaches the slow start threshold (ssthresh), typically half the value before the last loss event.
3. What is the role of the congestion window (cwnd) in TCP, and how is it adjusted? (Slide 3-93) **Your answer:** cwnd is dynamically adjusted in response to observed network congestion (implementing TCP congestion control) **Correction:** Correct! The congestion window (cwnd) limits the amount of unacknowledged data in the network and is dynamically adjusted based on network feedback (ACKs, losses) to control congestion.
4. How does TCP CUBIC differ from classic AIMD, and what advantage does it provide? (Slides 3-97 to 3-98) **Your answer:** TCP CUBIC sends a cubic increase (instead of additive), but it asymptotes before M-Decr. This sends more data than linear (additive). **Correction:** Correct! TCP CUBIC increases the congestion window using a cubic function, allowing for faster recovery to the previous maximum and more aggressive probing for bandwidth, especially in high-speed networks, compared to classic AIMD.
5. Why is the bottleneck link important in understanding TCP throughput? (Slides 3-99 to 3-100) **Your answer:** the bottleneck link is the maximum capacity (the weakest link in the chain) **Correction:** Correct! The bottleneck link determines the maximum achievable throughput for a TCP connection; sending faster than this rate only increases delay, not throughput.

Important Notes (Pages 91–100)

- MTT (Mean Transit Time) changes over time.
 - The bigger the packet, the longer the transit time.
 - Also understand piggybacking (combining data and ACKs in one segment).
- cwnd = Congestion window (see image below).

Congestion window

Congestion window

Summary of Pages 101–110

Delay-Based and Explicit Congestion Control; Fairness; QUIC

- Delay-based TCP congestion control aims to keep the sender-to-receiver path just full enough to maximize throughput while minimizing delay. It increases the congestion window (cwnd) if measured throughput is close to the uncongested rate, and decreases it if throughput drops, indicating congestion.
- This approach avoids inducing loss and is used in protocols like BBR (used by Google).
- Explicit Congestion Notification (ECN) allows routers to mark packets to signal congestion, with endpoints adjusting their behavior accordingly. ECN involves both IP and TCP header bits.
- TCP fairness: If multiple TCP sessions share a bottleneck, each should ideally get an equal share of bandwidth. AIMD helps achieve fairness under ideal conditions (same RTT, fixed number of sessions).
- Not all applications are “fair”—UDP-based apps (like multimedia) may ignore congestion control, and applications can open multiple parallel TCP connections to get more bandwidth.
- The transport layer is evolving: new protocols like QUIC (used for HTTP/3) run on top of UDP, providing features like multiplexed streams, integrated security, and improved performance. QUIC adopts many TCP-like mechanisms for congestion and error control, but with faster connection setup and more flexibility.

Slide Images (Pages 101–110)

Slides 101–110 are available as PNGs in the pdf_outputs folder:

slide-101.png, slide-102.png, ..., slide-110.png

Comprehension Q&A (Pages 101–110) – Answers & Corrections

1. How does delay-based TCP congestion control differ from loss-based approaches, and what is its main goal? (Slides 3-101 to 3-102) **Your answer:** you're no longer measuring loss of packets or timeout, but delay (RTT) **Correction:** Correct! Delay-based congestion control uses measured round-trip time (RTT) to infer congestion, aiming to keep the network path just full enough for high throughput but low delay, rather than relying on packet loss or timeouts as congestion signals.

2. What is Explicit Congestion Notification (ECN), and how does it work in TCP/IP networks? (Slide 3-103) **Your answer:** IP places an ECN header, and TCP places C,E header **Correction:** Correct! ECN uses bits in the IP header (ECN field) to signal congestion, and TCP uses the C and E bits in its header. Routers mark packets to indicate congestion, and endpoints adjust their behavior accordingly.
3. What is the fairness goal for TCP connections sharing a bottleneck link, and how does AIMD help achieve it? (Slides 3-104 to 3-105) **Your answer:** if K TCP sessions share same bottleneck link of bandwidth R, each should have average rate of R/K . AIMD is very constant in its increase/decrease. **Correction:** Correct! The fairness goal is for each of K TCP sessions to get an average rate of R/K . AIMD helps achieve this by increasing and decreasing each flow's rate in a coordinated way, so all flows converge to a fair share under ideal conditions.
4. Why might some applications not be "fair" in their use of network resources? (Slide 3-106) **Your answer:** either they use UDP, or applications open MULTIPLE parallel connections to trick the system **Correction:** Correct! Some applications use UDP to avoid congestion control, or open multiple parallel TCP connections to get more bandwidth, which can undermine fairness.
5. What are the key features and advantages of QUIC compared to traditional TCP? (Slides 3-108 to 3-110) **Your answer:** increase performance of HTTP, deployed on many Google servers & apps (Chrome, mobile YouTube app) **Correction:** Correct! QUIC improves HTTP performance, is widely deployed (e.g., Google, Chrome, YouTube), provides faster connection setup, multiplexed streams, integrated security, and uses UDP as its transport.

Summary of Pages 111–121

QUIC, Go-Back-N, TCP FSMs, and Throughput

- QUIC combines transport and security handshakes into one, reducing connection setup time compared to TCP+TLS. It supports parallel streams, avoiding head-of-line (HOL) blocking, and integrates reliability, congestion control, authentication, and encryption.
- Go-Back-N protocol: Sender can have up to N unacknowledged packets in flight. On timeout, all unacknowledged packets are retransmitted. Receiver only accepts in-order packets and always ACKs the highest in-order sequence number.
- TCP sender FSM: Manages sequence numbers, timers, retransmissions, and cumulative ACKs. Retransmits the lowest unacknowledged segment on timeout.
- TCP 3-way handshake FSM: Describes the states and transitions for connection setup (SYN, SYN-ACK, ACK) and teardown (FIN, ACK, CLOSE_WAIT, etc.).
- TCP throughput: Average throughput is determined by the window size (W) and RTT. For high-speed, long-distance links ("long, fat pipes"), achieving high throughput

requires a large window and very low loss rates. Specialized TCP versions exist for these scenarios.

Slide Images (Pages 111–121)

Slides 111–121 are available as PNGs in the pdf_outputs folder:

slide-111.png, slide-112.png, ..., slide-121.png

Comprehension Q&A (Pages 111–121) – Answers & Corrections

1. How does QUIC improve connection establishment compared to TCP+TLS? (Slide 3-111) **Your answer:** TCP (transport) and TLS (security) handshakes must be done in sequence, compared to (1x) QUIC handshake **Correction:** Correct! QUIC combines transport and security handshakes into a single step, reducing connection setup time compared to the sequential TCP and TLS handshakes.
2. What is the main advantage of QUIC's parallel streams over TCP? (Slide 3-112) **Your answer:** Quic has a great way of catching errors, moving-on, and retransmitting error packets when resources are available **Correction:** Partially correct. The main advantage is that QUIC supports multiple parallel streams within a single connection, so an error or delay in one stream does not block others (no head-of-line blocking), improving performance and reliability.
3. How does the Go-Back-N protocol handle lost or out-of-order packets? (Slides 3-115 to 3-116) **Your answer:** not relevant section **Correction:** Skipped as requested.
4. What are the key states in the TCP 3-way handshake and connection teardown? (Slides 3-118 to 3-119) **Your answer:** SYN, SYNACK, SYN rcvd, ESTAB **Correction:** Correct! Key states include SYN (sent), SYN-ACK (received), SYN rcvd, ESTAB (established), and states for connection teardown like FIN_WAIT, CLOSE_WAIT, LAST_ACK, and CLOSED.
5. How is average TCP throughput related to window size and RTT? (Slides 3-120 to 3-121) **Your answer:** not relevant section **Correction:** Skipped as requested.

Important Note (Pages 111–121)

- Latency is startup time

4

Summary of Chapter 4, Pages 1–29

Network Layer: Data Plane Overview

- The network layer provides services for packet delivery across networks, focusing on the data plane (forwarding) and control plane (routing).

- Data plane: Local, per-router function that forwards packets based on header values and forwarding tables.
- Control plane: Network-wide logic that determines routing paths, implemented via distributed algorithms or SDN (Software-Defined Networking).
- Routers use input ports, switching fabrics, and output ports to move packets efficiently. Input ports perform header lookup and forwarding decisions, often using longest prefix matching.
- Service models: The Internet uses a “best effort” model (no guarantees on delivery, order, or timing), while other architectures (ATM, Intserv, Diffserv) offer varying levels of QoS guarantees.
- Switching fabrics connect input and output ports using memory, bus, or interconnection networks (e.g., crossbar, multistage switches).
- Longest prefix matching and TCAMs are used for fast forwarding decisions in routers.

Slide Images (Chapter 4, Pages 1–29)

Slides 1–29 are available as PNGs in the pdf_outputs folder:
slide_ch4-1.png, slide_ch4-2.png, ..., slide_ch4-29.png

Comprehension Q&A (Chapter 4, Pages 1–29) – Answers & Corrections

1. What are the main functions of the network layer, and how do the data plane and control plane differ? (Slides 4-2 to 4-6) **Your answer:** transport the segment from sending to receiving host, forwarding & routing, Data plane (local), Control plane (network-wide) **Correction:** Correct! The network layer transports segments between hosts, with the data plane handling local forwarding and the control plane managing network-wide routing decisions.
2. What is the difference between forwarding and routing? (Slide 4-5) **Your answer:** Forwarding is router to router, routing is the process of host to host via router **Correction:** Partially correct. Forwarding is the process of moving packets from a router’s input to the appropriate output link; routing is the process of determining the end-to-end path through the network.
3. How does Software-Defined Networking (SDN) change the control plane compared to traditional routing? (Slide 4-8) **Your answer:** a remote controller computes & installs forwarding tables in routers **Correction:** Correct! SDN centralizes the control plane, with a remote controller computing and installing forwarding tables in routers, rather than each router running its own distributed algorithm.
4. What is the Internet’s “best effort” service model, and how does it compare to ATM or Intserv? (Slides 4-10 to 4-12) **Your answer:** Internet’s best effort means no guarantees on : 1) success 2) timing, or 3) bandwidth. It’s much worse than an ATM for BW, Loss, Order & Timing guarantees, and Intserv somehow provides internet guarantees in all of those areas. **Correction:** Correct! The Internet’s best effort

model provides no guarantees on delivery, timing, or bandwidth, while ATM and Intserv can provide various levels of guarantees for bandwidth, loss, order, and timing.

5. What is longest prefix matching, and why is it important in router forwarding? (Slides 4-18 to 4-22) **Your answer:** when looking for forwarding table entry for given destination addresses, use LONGEST address prefix that matches destination address. It's good for addressing, apparently. **Correction:** Correct! Longest prefix matching ensures the most specific route is chosen for a destination address, which is essential for efficient and accurate forwarding in routers.
6. Name and briefly describe the three main types of switching fabrics used in routers. (Slides 4-24 to 4-27) **Your answer:** memory, which uses assigned addresses. bus, which is hard-wired. interconnected network, which uses an array of connections to find src/dst. **Correction:** Partially correct. The three types are: memory (packets copied through shared memory), bus (packets transferred via a shared bus), and interconnection network (packets switched through a network of interconnected switches, e.g., crossbar or multistage).

Important Notes (Chapter 4, Pages 1–29)

- Random question from a network diagram: Which type of topology is this?
 - Physical
 - Logical
 - Because each are connected at the network layer, and the network is doing all of the work
- Note for Mikhail: Practice the multistage switch on page 28 & 29. Ask yourself: How about efficiency? Call blocking?
- Finally, it may be better to review your vocab sheet than even reading this document...